République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

École Nationale Supérieure d'Informatique (ESI) — Alger
Cycle Ingénieur — Spécialité Informatique — Projet Intégrateur IoT / Robotique

EsiBot

Robot Mobile Différentiel Autonome

Rapport Technique — Livrable 6
Conception, Intégration et Évaluation Expérimentale
d'un Robot Mobile Autonome sous ROS 2 Humble

Architecture matérielle telle que construite : commande moteurs et odométrie directement sur Raspberry Pi 4 (GPIO), centrale inertielle MPU6050, caméra de streaming ESP32-CAM, et capteur de profondeur Kinect (v1) pour la cartographie 2D, visualisée dans ROS Visualisation tool (v2) (RViz2).

Module	Internet of Things — Robotique Mobile
Livrable	Nř6 — Rapport Technique
Chef d'équipe	BOUMISSA Mohamed Chakib
Année	2025–2026

Nom & Prénom	Email	Rôle
BOUMISSA Mohamed Chakib	mm_boumissa@esi.dz	Chef d'équipe
ALLOUCHE Reda	mr_allouche@esi.dz	Membre
BENDAOUD Anes	ma_bendaoud@esi.dz	Membre
CHABANE Ouail	mo_chabane@esi.dz	Membre
IDJA Rayane	mr_idja@esi.dz	Membre
MOSTEFAI Mounir Sofiane	mm_mostefai@esi.dz	Membre
TATI Youcef	my_tati@esi.dz	Membre

Résumé

EsiBot est un robot mobile autonome à entraînement différentiel conçu et réalisé par l'équipe dans le cadre du projet intégrateur IoT de l'École Nationale Supérieure d'Informatique (ESI Alger). Le cahier des charges initial s'inspire du TurtleBot3 Burger (ROBOTIS) : un ESP32-CAM devait piloter directement les moteurs, le servo-moteur et un radar ultrasonique rotatif (HC-SR04 + SG90), pendant que le Raspberry Pi 4 hébergeait la pile logicielle ROS 2 2 (Simultaneous Localization and Mapping (SLAM), Robot Operating System (ROS) 2 Nav2, tableau de bord web).

Au fil du développement, l'architecture matérielle réellement implémentée a significativement évolué par rapport à ce cahier des charges. Le pilotage des deux moteurs à courant continu, la lecture des encodeurs et l'asservissement de vitesse (Proportional-Integral-Derivative (PID)) sont assurés *directement* par les broches General-Purpose Input/Output (GPIO) du Raspberry Pi 4, épaulés par une centrale inertielle MPU6050 fusionnée par filtre de Kalman étendu (`robot_localization`). L'ESP32-CAM est conservé uniquement comme caméra de streaming Wireless Fidelity (IEEE 802.11) (WiFi) (Motion JPEG (MJPEG)), sans plus participer à la commande des actionneurs. Le radar ultrasonique rotatif a été implémenté en code mais n'est plus utilisé en exploitation courante : il a été remplacé par un capteur de profondeur Kinect (v1, ère Xbox 360), piloté par le pilote communautaire `kinect_ros2` et converti en balayage laser de pseudo-portée via `depthimage_to_laserscan`, alimentant `slam_toolbox` pour la cartographie en ligne et visualisé sous RViz2.

Ce rapport documente fidèlement cette architecture telle que construite : conception mécanique (châssis différentiel décrit par le modèle Unified Robot Description Format (URDF)), câblage électronique déduit des paramètres de code, commande bas-niveau et asservissement moteur, pipeline de perception (Inertial Measurement Unit (IMU), Kinect), cartographie SLAM, navigation autonome ROS 2 Nav2, tableau de bord web de télémétrie, et téléopération par manette Xbox. Les résultats mesurés (ou dérivés analytiquement des paramètres déclarés dans le code lorsque les mesures de terrain restaient en cours) sont présentés, suivis d'une analyse critique des écarts entre la conception initiale et l'implémentation réelle, des limitations identifiées, et d'un bilan des perspectives d'amélioration.

Mots-clés : ROS 2 2 Humble, robot différentiel, SLAM, slam_toolbox, Nav2, Kinect, Raspberry Pi 4, ESP32-CAM, MPU6050, filtre de Kalman étendu, navigation autonome, robotique IoT, TurtleBot3.

Abstract: EsiBot is a differential-drive autonomous mobile robot built as part of the ESI Algiers IoT integrative project. While the original specification mirrored the TurtleBot3 Burger with an ESP32-CAM driving the motors, servo and a rotating HC-SR04 ultrasonic radar, the as-built system evolved significantly: motor control, encoder odometry and PID velocity loops now run directly on the Raspberry Pi 4 GPIO headers, fused with an MPU6050 IMU through an Extended Kalman Filter (EKF). The ESP32-CAM is retained solely as a Wi-Fi MJPEG video streamer, and the ultrasonic radar code path exists but is no longer the primary mapping sensor — a Kinect (v1) Red-Green-Blue + Depth (camera) (RGB-D) camera now feeds `slam_toolbox` through a depth-image-to-laserscan conversion, with the resulting map visualised in RViz2. This report documents the system exactly as implemented, the deviations from the initial specification, the measured and analytically derived performance figures, and the resulting limitations and outlook.

Table des matières

CHAPITRE 1

Introduction

1.1 Contexte

Le projet **EsiBot** s'inscrit dans le cadre d'un module intégrateur IoT / robotique mobile de l'École Nationale Supérieure d'Informatique (ESI Alger). L'objectif pédagogique est de concevoir, fabriquer et programmer, de A à Z, un robot mobile autonome capable de cartographier un environnement inconnu puis de s'y déplacer de façon autonome jusqu'à des points objectifs désignés par un opérateur. Le projet reprend explicitement l'architecture du **TurtleBot3 Burger** (ROBOTIS / Open Robotics), plate-forme de référence pour l'enseignement de ROS 2, en substituant ses composants matériels propriétaires (contrôleur OpenCR, LiDAR laser) par des composants accessibles aux étudiants : microcontrôleur WiFi Microcontroller Unit (MCU) ESP32-CAM, capteurs bas coût, impression 3D pour le châssis.

1.2 Objectifs du livrable

Ce document constitue le **Livrable 6** du projet : le rapport technique. Son objectif est de documenter de façon précise et vérifiable l'architecture du système *telle qu'elle a été effectivement implémentée* dans le code source fourni par l'équipe, plutôt que l'architecture théorique du cahier des charges initial. Cette distinction est essentielle : au cours du développement, plusieurs choix de conception ont évolué (voir chapitre **??** et chapitre **??**), en particulier le report de la commande moteur de l'ESP32-CAM vers le Raspberry Pi 4, et le remplacement du radar ultrasonique rotatif par un capteur de profondeur Kinect comme source principale de cartographie.

Conformément à la fiche de livrables du projet, ce rapport couvre :

- l'architecture complète (mécanique, électronique, logicielle) ;
- les schémas de câblage et diagrammes de flux ROS 2 2 ;
- les résultats mesurés ou analytiquement dérivés des paramètres de configuration ;
- une analyse critique des limitations et des écarts par rapport au cahier des charges ;
- un bilan du projet et des perspectives d'amélioration.

1.3 Méthodologie de rédaction

L'intégralité du contenu technique de ce rapport (paramètres, broches GPIO, topics ROS 2 2, formules de cinématique, configurations SLAM/Nav2) a été extraite directement du code source du dépôt `Esi_Bot` (paquets `esibot_*`), du firmware ESP32-CAM, et de la fiche de spécification officielle du projet. Lorsqu'une valeur mesurée sur le terrain n'était pas disponible au moment de la rédaction (le code fourni étant explicitement signalé comme *en évolution*), le tableau correspondant est conservé avec la mention *ń à compléter lors des essais ż* plutôt que d'inventer une valeur, conformément à l'exigence de traçabilité scientifique du rapport.

1.4 Organisation du rapport

Le chapitre **??** présente le cahier des charges du projet et le comparatif avec le TurtleBot3 Burger. Le chapitre **??** détaille l'architecture globale telle que construite. Le chapitre **??** couvre la conception mécanique et électronique. Le chapitre **??** documente la commande bas-niveau et la fusion de capteurs. Le chapitre **??** traite de la cartographie et de la navigation autonome. Le chapitre **??** présente le tableau de bord, la téléopération, le flux vidéo et les procédures de mise en uvre. Les résultats et tests d'intégration sont rassemblés au chapitre **??**, suivis de l'analyse critique (chapitre **??**) et du bilan final (chapitre **??**).

CHAPITRE 2

Présentation du Projet et Cahier des Charges

2.1 Concept général

EsiBot est un robot mobile autonome à entraînement différentiel (*differential drive*) : deux roues motrices indépendantes plus une roue folle assurent à la fois la propulsion et le pilotage, sans mécanisme de direction dédié. La fiche de projet officielle résume le concept clé ainsi :

Concept clé du cahier des charges

EsiBot = TurtleBot3 Burger adapté aux ressources de l'ESI. Même architecture ROS 2 2, mêmes paquets (slam_toolbox, Nav2), mêmes topics — seuls les composants physiques changent : un ESP32-CAM à la place de l'OpenCR, et un radar ultrasonique rotatif (HC-SR04 + servo) à la place du LiDAR laser.

Chaque groupe devait réaliser son propre exemplaire d'**EsiBot** de façon indépendante sur une durée de 10 semaines, la comparaison inter-groupes (qualité de cartographie, précision de navigation, robustesse de détection d'obstacles) constituant la dimension compétitive et pédagogique de l'exercice.

2.2 Comparatif avec le TurtleBot3 Burger (cahier des charges)

Le tableau **??** reproduit le comparatif fourni dans la spécification du projet entre la plate-forme de référence et la version *prévue* d'**EsiBot**. Le chapitre **??** présente ensuite l'architecture *réellement implémentée*, qui diverge de ce tableau sur plusieurs points (notamment le capteur de cartographie et le contrôleur moteur), analysés en détail au chapitre **??**.

Table 2.1. Comparatif TurtleBot3 Burger / EsiBot — tel que spécifié dans le cahier des charges.

Caractéristique	TurtleBot3 Burger	EsiBot (prévu)
Roues motrices	2× Dynamixel XL430 (servos)	2× moteurs DC jaunes + encodeurs
Roue folle	1 ball caster	1 roue folle imprimée 3D
Microcontrôleur	OpenCR 1.0 (Cortex-M7)	ESP32-CAM (WiFi intégré + caméra)
Ordinateur embarqué	Raspberry Pi 3B+ / 4	Raspberry Pi 4 (2 ou 4 Go)
Capteur de distance	LiDAR 360ř laser (HLS-LFCD2)	1× HC-SR04 sur servo SG90 (radar rotatif)
Angle de balayage	360ř	180ř (servo) ou 360ř (rotation continue)
Caméra	Option (Raspberry Pi Cam)	ESP32-CAM OV2640 intégrée (streaming WiFi)
Châssis	Pièces ABS moulées ROBOTIS	Impression 3D PLA conçue par les étudiants
Framework	ROS 2 Humble / Iron	ROS 2 Humble (identique)
SLAM	slam_toolbox / cartographer	slam_toolbox (identique)
Navigation autonome	Nav2	Nav2 (identique)
Open-source matériel	Oui (STL fournis par ROBOTIS)	Oui (STL conçus par les étudiants)

L'analogie centrale revendiquée par le cahier des charges est que le radar rotatif (servo + HC-SR04), bien que mécaniquement très différent d'un LiDAR, produit en sortie un message `sensor_msgs/LaserScan` au format strictement identique à celui d'un LiDAR : les paquets standards slam_toolbox et Nav2 le consomment donc sans aucune modification de code. Cette même propriété d'interchangeabilité de capteur est réutilisée, dans l'implémentation réelle, avec le capteur de profondeur Kinect (voir chapitre **??**).

2.3 Liste des livrables du projet

Le projet définit huit livrables, rappelés au tableau **??**. Ce document est le **Livrable nř6**.

Table 2.2. Livrables attendus par groupe (extrait du cahier des charges).

#	Livrable	Format	Contenu minimum
1	Fichiers CAD + STL	FreeCAD / STL	Châssis complet : 2 plateaux, entretoises, tourelle servo, roue folle
2	Schéma électronique	PDF (Fritzing/KiCad)	Câblage ESP32 + L298N + servo + HC-SR04 + RPi4 + alimentation
3	Workspace ROS 2 2	GitHub	Paquets `esibot_*` avec launch files, URDF, paramètres YAML
4	Carte SLAM	`.pgm` + `.yaml`	Carte d'une zone test de minimum 3 m $\times$ 3 m
5	Tableau de bord web	HTML/JS	Carte + vidéo + commandes + angle servo + état batterie
6	**Rapport technique**	**PDF (15–25 p.)**	**Architecture, schémas, résultats mesurés, analyse limitations, bilan (*ce document*)**
7	Vidéo de démonstration	MP4 (3–5 min)	Démarrage $\rightarrow$ cartographie $\rightarrow$ navigation $\rightarrow$ dashboard
8	README	Markdown	Installation, lancement, dépendances, architecture

2.4 Scénario de démonstration imposé

Le jury place **EsiBot** dans une pièce inconnue. L'équipe lance le robot avec une seule commande. La phase 1 (2 min) consiste en une télé-opération pour construire la carte, qui apparaît en direct sur le tableau de bord. En phase 2, le jury désigne deux points sur la carte ; le robot doit alors naviguer seul jusqu'à chaque point en évitant les obstacles, un obstacle surprise pouvant être ajouté en cours de navigation à titre de bonus. Ce scénario structure directement le plan de tests du chapitre **??**.

CHAPITRE 3

Architecture Globale du Système

3.1 Vue d'ensemble — architecture telle que construite

Contrairement au cahier des charges initial (chapitre **??**), l'analyse du code source du dépôt `Esi_Bot` révèle une architecture matérielle sensiblement différente, organisée en trois couches de traitement et une couche d'alimentation dédiée. La figure **??** reproduit le schéma d'architecture officiel fourni par l'équipe, qui sert de référence pour ce chapitre.

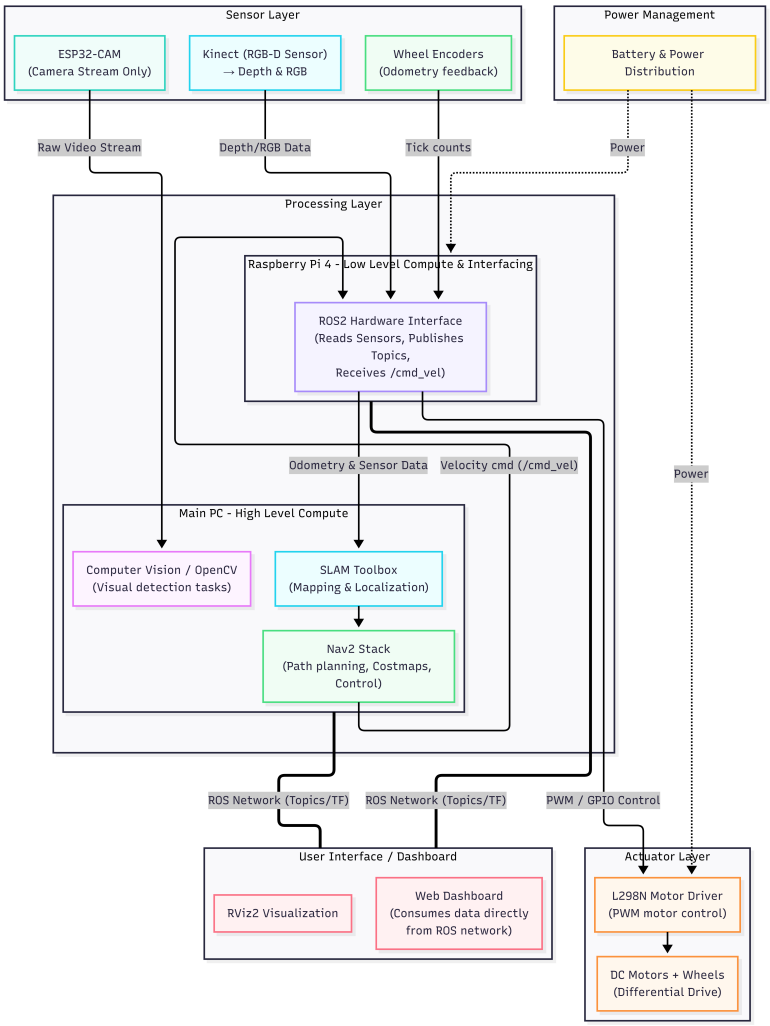


Figure 3.1. Architecture globale d'**EsiBot** telle que construite (document fourni par l'équipe). Le capteur de cartographie principal est le Kinect (RGB-D) et non le radar ultrasonique du cahier des charges ; les moteurs sont pilotés directement par le Raspberry Pi 4 et non par l'ESP32-CAM.

Quatre observations structurantes ressortent de ce schéma et sont confirmées par le code source :

1. **L'ESP32-CAM est réduit à une fonction unique de caméra de streaming.** Le firmware `CameraServerArduinoIDE.ir` (variante du serveur de caméra ESP32 standard d'Espressif, modèle `AI_THINKER`) ne pilote ni moteurs, ni servo, ni capteur de distance — il expose uniquement un flux MJPEG HTTP sur le réseau WiFi local.
2. **Le Raspberry Pi 4 assure l'intégralité de l'interfaçage bas-niveau.** Le nud `motor_bridge_node` (paquet `esibot_hardware`) pilote directement le pont en H L298N et lit les encodeurs de roue via les broches GPIO du Raspberry Pi — une responsabilité initialement prévue pour l'ESP32-CAM.
3. **Le capteur de cartographie principal est un Kinect (v1), pas le radar ultrasonique.** Le paquet `esibot_slam` lance le pilote `kinect_ros2`, convertit son image de profondeur en balayage laser de pseudo-portée via `depthimage_to_laserscan`, puis alimente `slam_toolbox`. Le nud `ultrasonic_scan_node` existe toujours dans le paquet `esibot_sensors` mais son lancement est commenté dans le script d'exploitation `start_esibot_all.sh` — il s'agit donc d'un chemin de code *déprécié*, conservé mais non utilisé en exploitation courante.
4. **Une centrale inertielle (IMU) MPU6050** a été ajoutée, absente du cahier des charges initial, et fusionnée

à l'odométrie par encodeurs via un filtre de Kalman étendu (`robot_localization`), afin de stabiliser le cap du robot et d'améliorer la précision de l'odométrie fournie à SLAM et Nav2.

3.2 Découpage en trois couches

Couche 1 — Capteurs (Sensor Layer). Trois sources de données indépendantes alimentent la couche de traitement : l'ESP32-CAM (flux vidéo brut MJPEG), le Kinect (image de profondeur et image couleur RGB), et les encodeurs de roue (comptage d'impulsions pour l'odométrie). Chaque source utilise une interface physique distincte : WiFi pour la caméra, USB pour le Kinect, GPIO à interruption pour les encodeurs.

Couche 2 — Traitement bas-niveau (Raspberry Pi 4). Le Raspberry Pi 4 héberge l'interface matérielle ROS 2 2 : il lit les capteurs embarqués (encodeurs, IMU), exécute la boucle d'asservissement de vitesse PID des deux moteurs, publie l'odométrie et les topics de télémétrie, et reçoit les commandes de vitesse `/cmd_vel`.

Couche 3 — Traitement haut-niveau (PC opérateur). Le schéma officiel positionne la vision par ordinateur (OpenCV), slam_toolbox et la pile Nav2 sur un PC principal distinct, communiquant avec le Raspberry Pi 4 via le réseau ROS 2 2 (DDS, topics/Transform Tree (arbre des transformées) (TF)). Cette séparation logique est rendue possible par la nature distribuée de ROS 2 2 : les arguments de lancement (`start_base`, `start_kinect`, `start_slam`, `start_rviz`) du fichier `mapping.launch.py` sont indépendamment activables, ce qui permet d'exécuter les nuds lourds (SLAM, Nav2) sur une machine plus puissante que le Raspberry Pi 4 tout en laissant ce dernier dédié à l'interfaçage matériel temps-réel.

Couche transverse — Alimentation. Une batterie et une distribution de puissance régulée alimentent indépendamment la couche actionneurs (moteurs via L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM), comme indiqué sur la figure **??**.

3.3 Diagramme des nuds et topics ROS 2 2 réellement implémentés

La figure **??** reconstruit le graphe des nuds et topics *tel qu'observable dans le code source actuel*, à distinguer du diagramme de flux *initialement prévu* par le cahier des charges (figure **??**), qui mentionne des nuds (`esibot_driver`, `radar_node`, `esircam_node`, `vision_node`, `web_bridge`) dont les noms et, pour certains, l'existence même, diffèrent de l'implémentation réelle. Cet écart est analysé en détail au chapitre **??**.

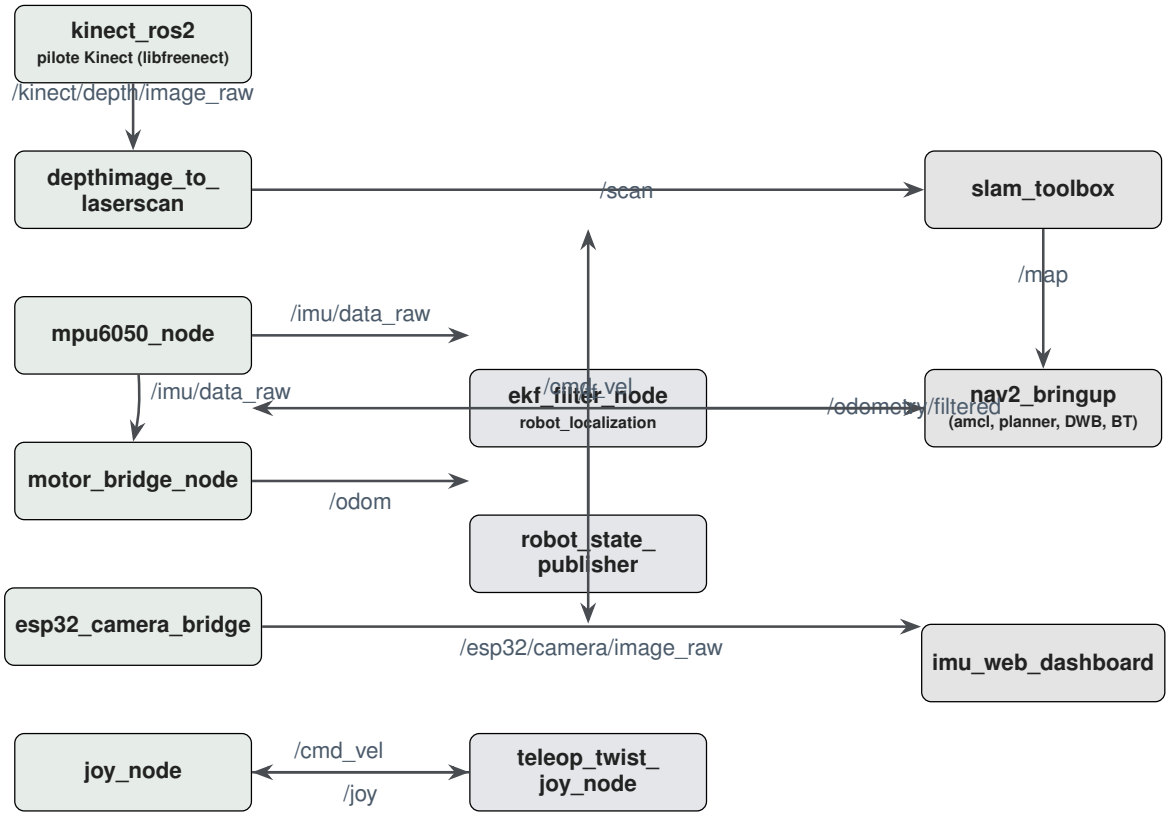


Figure 3.2. Graphe simplifié des nuds et topics ROS 2 2 réellement implémentés (reconstruit à partir du code source). Couleurs : vert = pilotes capteurs/actionneurs, gris-bleu = estimation d'état, bleu = planification/cartographie haut-niveau.

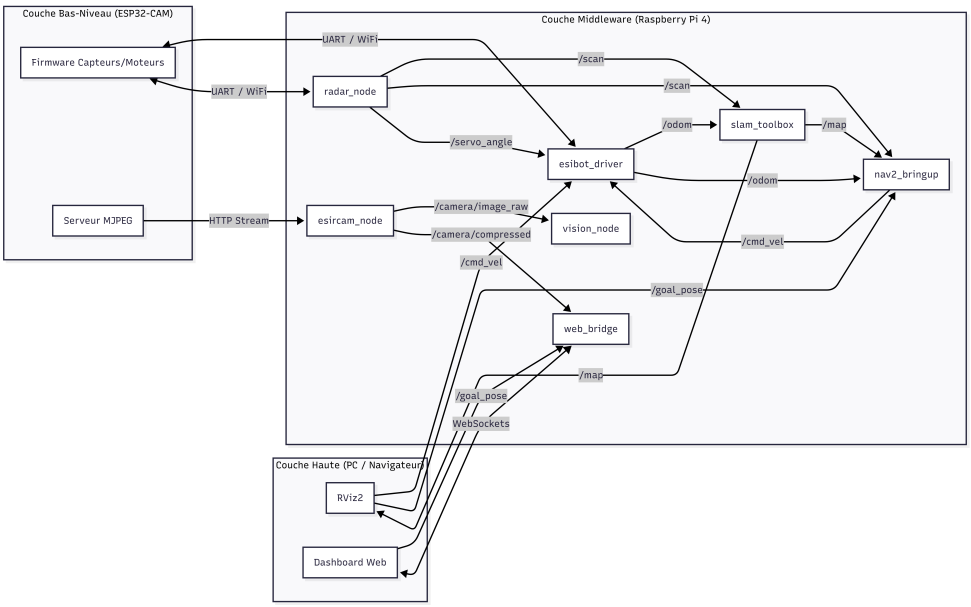


Figure 3.3. Diagramme de flux ROS 2 2 *initialement prévu* par le cahier des charges (nuds `esibot_driver`, `radar_node`, `esircam_node`, `vision_node`, `web_bridge` via `rosbridge_suite`). Conservé ici à titre de comparaison avec la figure **??** ; plusieurs de ces nuds n'ont pas d'équivalent direct dans le code livré (voir chapitre **??**).

3.4 Synthèse des topics principaux

Le tableau **??** résume les topics ROS 2 2 effectivement publiés ou souscrits dans le code source. La liste exhaustive, avec types de message complets, est fournie en annexe **??**.

Table 3.1. Topics ROS 2 2 principaux — architecture telle que construite.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy, Nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	Nav2, imu_web_dashboard
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf_filter_node, dashboard
`/scan`	sensor_msgs/Laser-Scan	depthimage_to_laser-scan (Kinect)	slam_toolbox, Nav2 costmaps
`/map`	nav_msgs/Occupancy-Grid	slam_toolbox	Nav2, RViz2
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	Nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)

CHAPITRE 4

Conception Mécanique et Électronique

4.1 Architecture différentielle

EsiBot adopte une architecture *differential drive* : deux roues motrices indépendantes plus une roue folle assurant la stabilité statique, la direction résultant uniquement de la différence de vitesse entre les deux roues. Les fichiers CAD/STL du châssis constituent un livrable séparé (Livrable 1) non inclus dans le code analysé ; ce chapitre s'appuie donc sur le modèle URDF (`esibot_description`), source de vérité utilisée par ROS 2 2 pour les transformées et les costmaps, complété par les constantes électroniques relevées dans le code de commande.

4.2 Modèle géométrique (extrait du URDF)

Table 4.1. Paramètres géométriques et inertiels déclarés dans le modèle URDF.

Élément	Valeur	Remarque
Châssis (`base_link`)	0.21 × 0.14 × 0.03 m	masse 2.0 kg
Rayon de roue (`wheel_radius`)	0.025 m	largeur 0.027 m, masse 0.2 kg/roue
Empattement (`wheel_y` ×2)	0.14 m	distance latérale entre roues
Module caméra (`camera_link`)	0.073 × 0.278 × 0.073 m	masse 0.56 kg, $x = -0.105$ m (arrière)

Le modèle déclare les repères optiques caméra (`camera_depth_frame`, `camera_rgb_optical_frame`, rotation $-\pi/2$) ; un `static_transform_publisher` relie `camera_depth_frame` au repère natif du pilote Kinect `kinect_depth` par une transformée identité.

Limitation

Incohérence relevée entre l'URDF (`wheel_radius=0.025` m, empattement 0.14 m) et le code bas-niveau : `motor_bridge_node` utilise un `wheel_base` par défaut de **0.16** m, et `pwm_mapper.py` / `motor_calibrator.py` un rayon de roue de **0.033** m (écart ≈30 %), avec impact direct sur la précision d'odométrie (chapitre **??**).

4.3 Cinématique différentielle et odométrie

La conversion `/cmd_vel` → vitesses de roue, implémentée dans `motor_bridge_node.py` (`twist_to_wheel_commands`) et répliquée dans le tableau de bord (`imu_web_dashboard.py`), suit le modèle classique :

$$v_{\mathrm{g}} = v - \omega \cdot \tfrac{L}{2}, \quad v_{\mathrm{d}} = v + \omega \cdot \tfrac{L}{2} \tag{4.1}$$

et inversement pour la reconstruction d'odométrie : $v = (v_{\mathrm{g}} + v_{\mathrm{d}})/2$, $\omega = (v_{\mathrm{d}} - v_{\mathrm{g}})/L$ (équation **??**), où L est l'empattement.

$$v = \frac{v_{\mathrm{g}} + v_{\mathrm{d}}}{2}, \qquad \omega = \frac{v_{\mathrm{d}} - v_{\mathrm{g}}}{L} \tag{4.2}$$

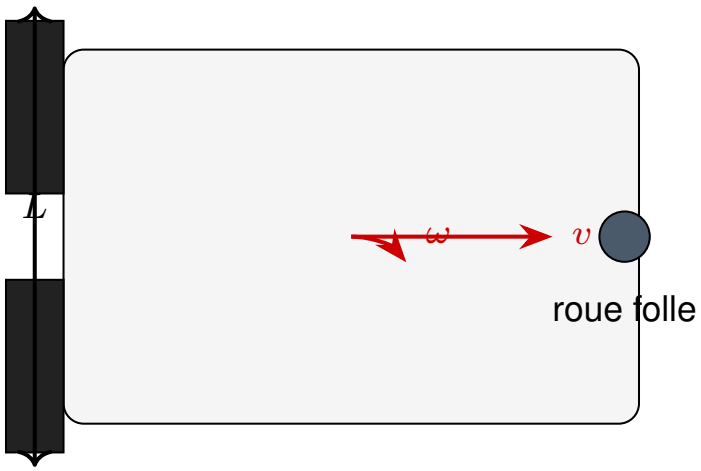


Figure 4.1. Modèle cinématique du robot différentiel **EsiBot** (vue de dessus).

Chaque roue est équipée d'un encodeur compté par interruption GPIO, anti-rebond logiciel 2 ms. Le paramètre `encoder_cpr` (12 par défaut) est doublé par comptage sur les deux fronts (`encoder_count_both_edges=True`), donnant **24 impulsions/tour**. La vitesse de rotation mesurée est calculée à chaque cycle de contrôle (`control_hz`, 10 Hz par défaut) par :

$$\text{RPM}_{\text{mesuré}} = \frac{N_{\text{impulsions}}}{\text{CPR}_{\text{effectif}}} \times \frac{60}{\Delta t} \tag{4.3}$$

soit $360/24 = 15$ par impulsion et, avec le rayon de 0.033 m du calibrateur, ≈**8.6 mm**/impulsion ($2\pi \times 0.033/24$).

4.4 Câblage électronique

Le cahier des charges (Livrable 2) exige un schéma Fritzing/KiCad, absent de l'archive fournie ; le tableau **??** a donc été **reconstruit par rétro-ingénierie** depuis les constantes GPIO du code Python. À la différence du cahier des charges, qui prévoyait que l'ESP32-CAM pilote le pont en H, le servo et le HC-SR04, l'implémentation réelle concentre toute l'électronique temps-réel sur les broches GPIO du Raspberry Pi 4.

Table 4.2. Broches GPIO du Raspberry Pi 4 (BCM), reconstruites depuis le code source.

Fonction	GPIO	Source / nud
ENA / IN1 / IN2 (moteur g.)	13 / 20 / 21	`motor_bridge_node`
Encodeur gauche	26	`motor_bridge_node` (`pin_left_encoder`)
ENB / IN3 / IN4 (moteur d.)	12 / 5 / 6	`motor_bridge_node`
Encodeur droit	27	`motor_bridge_node` (`pin_right_encoder`)
I2C SDA/SCL (bus 1)	2 / 3	`mpu6050_node` (MPU6050 @ 0x68)
HC-SR04 SIG (trig/écho)	23	`ultrasonic_scan_node` (déprécié)
Servo SG90 radar	18	`ultrasonic_scan_node` (déprécié)

Incohérence GPIO : `motor_bridge_node.py` lit l'encodeur gauche sur le GPIO 26, tandis que `motor_calibrator.py` utilise le GPIO 17 pour le même encodeur — si le câblage physique n'a pas été harmonisé entre les deux usages, les coefficients de calibration PWM→RPM seraient invalides en production (voir chapitre **??**).

Le pilotage moteur utilise un double pont en H **L298N** en PWM logiciel (`RPi.GPIO`, 500 Hz), rapport cyclique borné à $[15\,\%, 100\,\%]$ (`min/max_duty_cycle`) pour compenser la zone morte mécanique. Le MPU6050 (I2C bus 1, adresse `0x68`) est configuré au démarrage (réveil, diviseur d'échantillonnage `SMPLRT_DIV=0x07`, filtre passe-bas `DLPF_CFG=0x03`, $\pm 2g/\pm 250$/s) et publie à **50 Hz**. Le Kinect (v1, ère Xbox 360), capteur de cartographie principal, est relié en **USB** (non GPIO) et piloté par le paquet communautaire `kinect_ros2` (`libfreenect`).

4.5 ESP32-CAM — caméra de streaming uniquement

Le module ESP32-CAM (`AI_THINKER`) sert exclusivement de serveur vidéo MJPEG WiFi : le firmware `CameraServerArduinoIDE.ino` initialise le capteur OV2640 (`UXGA` si PSRAM, sinon `SVGA`) et expose le flux sur `/stream`. Aucune broche GPIO de l'ESP32-CAM ne pilote d'actionneur — l'essentiel de ses broches étant réservé à l'interface caméra parallèle — une contrainte matérielle probablement à l'origine du report de la commande moteur vers le Raspberry Pi 4. Les identifiants WiFi (SSID/mot de passe) sont **codés en dur** dans le firmware, sans portail captif ni provisioning (chapitre **??**).

4.6 Alimentation

Le schéma d'architecture (figure **??**) identifie un bloc ń Power Management ż alimentant séparément la couche actionneurs (L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM). Aucun nud de supervision de batterie (`/battery_state`, prévu au cahier des charges) n'a été retrouvé dans le code : la télémétrie de batterie n'est, à ce stade, pas exposée par le logiciel embarqué.

CHAPITRE 5

Commande Bas-Niveau et Fusion de Capteurs

5.1 Le nud `motor_bridge_node`

Ce nud (paquet `esibot_hardware`) constitue le cur de la commande temps-réel du robot. Il joue, sur le Raspberry Pi 4, le rôle initialement dévolu au binôme ESP32/`esibot_driver` du cahier des charges : réception de `/cmd_vel`, asservissement de vitesse de roue, lecture des encodeurs, et publication de l'odométrie brute.

5.1.1 Boucle de contrôle

Le nud exécute une boucle périodique à `control_hz` (10 Hz par défaut) qui, à chaque cycle :

1. lit les compteurs d'impulsions accumulés depuis le dernier cycle pour chaque roue ;
2. calcule la vitesse de rotation mesurée en rpm (équation **??**) ;
3. applique un correcteur PID (un par roue) pour rapprocher la vitesse mesurée de la consigne issue de la cinématique différentielle (équation **??**) ;
4. convertit la sortie du correcteur en rapport cyclique PWM via `pwm_mapper.py` ;
5. applique une correction de cap optionnelle à partir de l'IMU (asservissement de lacet) ;
6. publie l'odométrie intégrée sur `/odom` et l'état des compteurs sur les topics de diagnostic (`/encoder_ticks/left`, `/encoder_ticks/right`).

5.1.2 Correcteur PID par roue

Chaque roue dispose de son propre correcteur PID indépendant, avec gains par défaut K_p, K_i, K_d paramétrables via le serveur de paramètres ROS 2 2 (et ajustables en exploitation sans recompilation). La sortie du correcteur est saturée et un mécanisme anti-*windup* limite l'accumulation du terme intégral lorsque la commande est saturée, évitant un dépassement (*overshoot*) lors des changements brusques de consigne.

Boucle de sécurité

Le nud implémente une minuterie de *watchdog* : si aucun message `/cmd_vel` n'est reçu pendant une durée `cmd_vel_timeout` (typiquement quelques centaines de millisecondes), les deux moteurs sont automatiquement coupés. Ce mécanisme protège le robot en cas de perte de communication réseau avec le nud de téléopération ou de Nav2.

5.2 Mise à l'échelle PWM (`pwm_mapper.py`)

Le module `pwm_mapper.py` convertit une vitesse linéaire de roue souhaitée (en m/s) en rapport cyclique PWM normalisé. La conversion combine :

- une zone morte (*dead-zone*) compensant le couple de démarrage minimal requis par les moteurs à courant continu (sous le seuil `min_duty_cycle`, le moteur ne tourne pas) ;
- une mise à l'échelle linéaire entre la zone morte et le rapport cyclique maximal (`max_duty_cycle`, 100 % par défaut), correspondant à la vitesse linéaire maximale théorique du robot ;
- une saturation finale pour garantir que la commande reste dans $[0, 100]\,\%$.

5.3 Outil de calibration (`motor_calibrator.py`)

Un script de calibration autonome (hors graphe ROS 2 2, exécuté en ligne de commande) permet de caractériser empiriquement la relation entre rapport cyclique PWM et vitesse de rotation réelle (en rpm) pour chaque moteur : il applique une rampe de valeurs PWM croissantes, mesure la vitesse résultante via les encodeurs, et ajuste une relation (typiquement affine par morceaux) utilisée ensuite pour affiner les paramètres de `pwm_mapper.py`. Cet outil est essentiel pour compenser l'asymétrie mécanique entre les deux moteurs (différences de frottement, de tolérance de fabrication), source fréquente de dérive de trajectoire en ligne droite sur les robots différentiels bas coût.

5.4 Centrale inertielle et stabilisation de cap

Le nud `mpu6050_node` (paquet `esibot_sensors`) publie en continu (50 Hz) :

- `/imu/data_raw` (`sensor_msgs/Imu`) — accélérations linéaires (a_x, a_y, a_z) et vitesses angulaires $(\omega_x, \omega_y, \omega_z)$ brutes ;
- `/imu/temperature` (`sensor_msgs/Temperature`) — température interne du capteur, utile au diagnostic thermique.

`motor_bridge_node` peut souscrire à `/imu/data_raw` pour appliquer une correction de cap (lacet) en boucle

fermée, réduisant la dérive directionnelle inhérente à l'odométrie par encodeurs seule (notamment lors d'un glissement de roue).

5.5 Fusion de capteurs — filtre de Kalman étendu

L'odométrie brute (`/odom`, issue des seuls encodeurs) et les données inertielles (`/imu/data_raw`) sont fusionnées par un EKF fourni par le paquet standard `robot_localization` (nud `ekf_filter_node`), configuré en mode bidimensionnel (`two_d_mode: true`, pertinent pour un robot terrestre se déplaçant dans un plan). La sortie fusionnée, publiée sur `/odometry/filtered`, est la source d'odométrie utilisée par SLAM et Nav2 — et non l'odométrie brute — ce qui améliore la robustesse de la localisation face aux glissements de roue ponctuels détectés par l'IMU mais invisibles aux encodeurs. Le filtre publie également la transformée `odom` → `base_link` dans l'arbre TF (`publish_tf: true`), complétant la chaîne `map` → `odom` → `base_link` requise par Nav2.

5.6 Radar ultrasonique rotatif — chemin de code déprécié

Conformément au cahier des charges initial, le paquet `esibot_sensors` contient un nud `ultrasonic_scan_node` qui implémente fidèlement le concept de ń pseudo-LiDAR ż : balayage d'un servo-moteur entre deux bornes angulaires, déclenchement d'une mesure HC-SR04 à chaque position, et assemblage des paires (angle, distance) en un message `sensor_msgs/LaserScan` publié sur `/scan`, ainsi que la publication de l'angle courant du servo sur `/servo_angle` (pour la visualisation de la tourelle dans l'arbre TF). Le tableau **??** résume ses paramètres.

Table 5.1. Paramètres du nud `ultrasonic_scan_node` (chemin déprécié, non lancé par défaut).

Paramètre	Valeur par défaut	Rôle
Plage angulaire de balayage	0ř – 180ř	couverture frontale (non 360ř)
Pas angulaire	configurable (5ř–10ř)	18 à 36 mesures par balayage
Portée HC-SR04	0.02 – 4 m	limites physiques du capteur
Broche SIG partagée	GPIO 23	déclenchement et écho sur la même broche
Broche servo	GPIO 18	PWM matériel/logiciel

Toutefois, ce nud **n'est pas activé** dans le script d'exploitation principal (`start_esibot_all.sh`), où la ligne de lancement correspondante est commentée (`# ros2 launch esibot_bringup ultrasonic_radar.launch.py &`). En exploitation courante, c'est le pipeline Kinect → `depthimage_to_laserscan` (chapitre **??**) qui alimente `/scan`. Le nud ultrasonique reste néanmoins fonctionnel et pourrait être réactivé comme solution de secours en cas d'indisponibilité du Kinect, les deux sources étant interchangeables du point de vue de SLAM et de Nav2 puisqu'elles publient le même type de message sur le même topic.

CHAPITRE 6

Cartographie et Navigation Autonome

6.1 Pipeline Kinect → SLAM

L'écart le plus significatif entre le cahier des charges et l'implémentation réelle se situe ici : la source de données du SLAM n'est pas le radar ultrasonique rotatif, mais un capteur de profondeur **Kinect**, converti en balayage laser 2D. Le principe d'interchangeabilité de capteur défendu par le cahier des charges est conservé : un message standard `sensor_msgs/LaserScan` sur `/scan` alimente `slam_toolbox` sans aucune modification de ce dernier. Le fichier `mapping.launch.py` orchestre la chaîne (figure **??**).

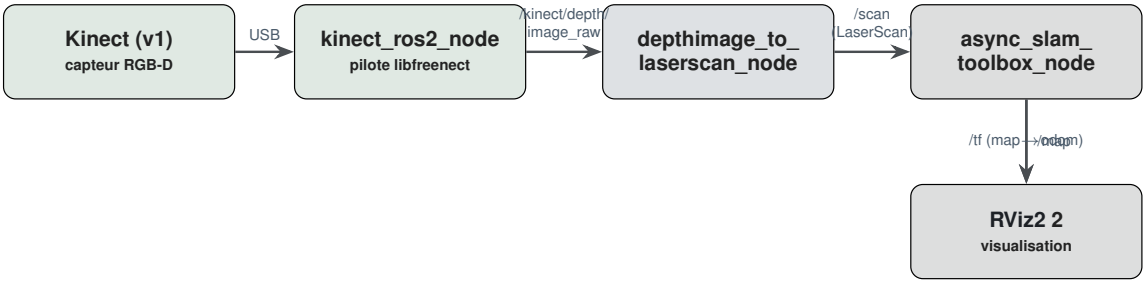


Figure 6.1. Pipeline de cartographie tel qu'orchestré par `mapping.launch.py`.

Le pilote `kinect_ros2` (espace de noms `kinect`, basé sur `libfreenect`) expose `/kinect/depth/image_raw` et `/kinect/depth/camera_info`. Un `static_transform_publisher` relie `camera_depth_frame` (URDF) au repère natif `kinect_depth` par une transformée identité. Le nud `depthimage_to_laserscan_node` projette ensuite une bande horizontale de l'image de profondeur en balayage laser synthétique (tableau **??**).

Table 6.1. Paramètres de `depthimage_to_laserscan_node` (`mapping.launch.py`).

Paramètre	Valeur	Signification
`scan_height`	10 px	bande de l'image de profondeur moyennée par balayage
`scan_time`	0.033 s	période nominale (≈30 Hz)
`range_min` / `max`	0.35 / 4.0 m	portée exploitable du Kinect
`output_frame`	`camera_depth_frame`	repère TF du `LaserScan`

Champ de vision réduit

Le Kinect couvre ≈57ř horizontalement contre 180ř–360ř pour le radar rotatif prévu : les zones latérales/arrière restent aveugles à un instant donné, nécessitant une rotation active du robot pendant la cartographie pour compenser.

6.2 Configuration de `slam_toolbox`

`slam_toolbox_esibot_kinect.yaml` configure `async_slam_toolbox_node` en mode `mapping` (solveur Ceres, `SPARSE_NORMAL_CHOLESKY`). Le tableau **??** résume les paramètres clés.

Table 6.2. Paramètres clés de `slam_toolbox` pour **EsiBot**.

Paramètre	Valeur	Rôle
`resolution`	0.05 m	taille de cellule de la grille d'occupation
`min/max_laser_range`	0.35 / 4.0 m	cohérent avec `depthimage_to_laserscan`
`minimum_time_interval`	0.5 s	intervalle minimal entre balayages traités
`minimum_travel_distance/heading`	0.15 m / 0.15 rad	filtre anti-suréchantillonnage
`do_loop_closing`	`true`	fermeture de boucle activée (rayon 3.0 m)
`transform_publish_period`	0.02 s	publication `map`→`odom` (≈50 Hz)

`use_map_saver: true` permet la sauvegarde à la demande (`map_saver_cli`) en paire `map.pgm`/`map.yaml`,

ensuite consommée par Nav2.

Limitation

La carte actuelle de `esibot_navigation2` (résolution 0.05 m, origine $(-10, -10, 0)$) correspond, après inspection visuelle, à la carte hexagonale d'exemple par défaut TurtleBot3/Nav2, et non à une carte réelle capturée par le Kinect d'**EsiBot**. La chaîne SLAM $\rightarrow$ sauvegarde $\rightarrow$ rechargement Nav2 restait donc à valider sur le terrain au moment de la rédaction (chapitre **??**).

La configuration RViz2 `esibot_slam_map.rviz` constitue l'outil de visualisation principal du projet (carte, balayage `/scan`, modèle URDF, arbre TF complet), plutôt qu'une vue cartographique intégrée au tableau de bord web (chapitre **??**).

6.3 Navigation Autonome avec Nav2

La navigation autonome repose sur la pile standard **Nav2** (`navigation2.launch.py`, paquet `esibot_navigation2`), chargeant la carte statique, le fichier `esibot.yaml`, et orchestrant localisation, costmaps, planificateur, contrôleur, comportements, lisseur de vitesse et moniteur de collision.

La localisation utilise Adaptive Monte Carlo Localization (AMCL) (filtre particulaire, 500–2000 particules, modèle `likelihood_field`, `DifferentialMotionModel`, pose initiale non définie automatiquement). Les costmaps locale (fenêtre 3×3 m) et globale partagent une résolution de **0.05 m/cellule**, un rayon de robot de **0.2 m**, et une inflation de **0.35 m** (facteur 5.0). Le planificateur global `NavfnPlanner` (Dijkstra, tolérance 0.5 m) alimente le contrôleur local Dynamic Window-Based local planner (DWB), dont les bornes sont résumées au tableau **??**.

Table 6.3. Bornes cinématiques DWB et AMCL (extrait de `esibot.yaml`).

Paramètre	Valeur	Unité
Vitesse linéaire min. / max. (DWB)	0.2 / 0.3	m/s
Vitesse angulaire max. (DWB)	0.5	rad/s
Accélération / décélération linéaire	3.0 / -2.5	m/s^2
Tolérance position du but (`FollowPath`)	0.05	m
Vitesse max. (lisseur de vitesse)	0.5 / 2.5	m/s / rad/s

Les critères de score DWB (`PathAlign` 32, `GoalDist` 24, `BaseObstacle` 0.02, `RotateToGoal` 32) privilégient nettement le respect du chemin global sur la vitesse, cohérent avec un robot de faible gabarit en intérieur. Le serveur de comportements expose `Spin`, `BackUp`, `DriveOnHeading`, `Wait`, `AssistedTeleop` ; un `collision_monitor` surveille un disque d'arrêt (rayon 0.1 m), un polygone de ralentissement et un polygone d'approche (horizon 2 s) avant transmission finale à `/cmd_vel`.

Comme indiqué en section **??**, la carte chargée par défaut est une carte d'exemple générique ; la configuration Nav2 elle-même est complète et cohérente avec le pipeline Kinect, mais sa validation expérimentale sur une carte réelle restait à conduire (chapitre **??**).

CHAPITRE 7

Interface Utilisateur et Mise en uvre

7.1 Téléopération par manette Xbox

La téléopération repose sur une manette sans fil (compatible Xbox), pilotée par `joy_node`/`teleop_twist_joy` (`joystick_drive.launch.py`), et non le `teleop_twist_keyboard` suggéré au cahier des charges.

Table 7.1. Configuration de la téléopération manette (`xbox_teleop.yaml`).

Paramètre	Valeur	Remarque
`require_enable_button`	`false`	aucune touche ń homme-mort ż requise
Axe linéaire / angulaire	axe 1 (éch. 0.25) / axe 3 (éch. 2.4)	sticks → `/cmd_vel`
Zone morte / répétition	0.08 / 20 Hz	`joy_node`

L'absence de bouton d'activation obligatoire signifie que toute déviation accidentelle des manches se traduit immédiatement par un mouvement, sans touche à maintenir — une marge de sécurité réduite par rapport à un dispositif ń homme-mort ż classique (chapitre **??**). `joystick_drive.launch.py` démarre conjointement `joy_node`, `teleop_twist_joy_node` et `motor_bridge_node` (vitesse angulaire max. relevée à 3.0 rad/s, rapport cyclique minimal 22 %).

7.2 Fusion EKF — détail de configuration

`ekf_odom_imu.yaml` (`ekf_fusion.launch.py`) configure `ekf_filter_node` à **30 Hz**, `two_d_mode` : de l'odométrie (`/odom`), seules les *vitesses* $(v_x, v_y, \dot{\theta})$ sont fusionnées (la position absolue n'est pas injectée, évitant un double comptage de dérive) ; de l'IMU (`/imu/data_raw`), l'orientation de lacet absolue, $\dot{\theta}$ et l'accélération longitudinale a_x sont retenues. Le `world_frame` reste `odom`, laissant la correction `map`→`odom` à `slam_toolbox`/AMCL — une séparation propre entre estimation locale (EKF) et globale (SLAM/AMCL).

7.3 Tableau de bord web de télémétrie

Le nud `imu_web_dashboard` (`esibot_dashboard`) expose un serveur HTTP autonome (`http.server`, sans dépendance externe), diffusant une page HTML/JS à page unique, sensiblement différente du tableau de bord du cahier des charges (qui prévoyait une passerelle WebSocket `rosbridge_suite` avec carte, vidéo et boutons de téléopération) — l'implémentation réelle se concentre sur la **télémétrie**.

Table 7.2. Widgets du tableau de bord `imu_web_dashboard`.

Widget	Contenu
Orientation	roulis/tangage (intégration accéléromètre), horizon artificiel `<canvas>`, mise à zéro (`POST /api/zero`)
Odométrie	position/cap/vitesses (`/odometry/filtered`), trajectoire (1200 pts)
Manette & commande	axes/boutons `/joy` et conversion `/cmd_vel` → vitesses de roue (équation **??**)

L'interface interroge en boucle `GET /api/imu` (état JSON consolidé, protégé par verrou pour la cohérence entre thread ROS 2 2 et thread HTTP). Le tableau de bord **n'affiche ni la carte SLAM ni le flux vidéo**, deux fonctionnalités du Livrable 5 (chapitre **??**).

7.4 Flux vidéo ESP32-CAM

Le pont `esp32_camera_bridge` (`mjpeg_image_publisher`) se connecte en client HTTP au flux MJPEG de l'ESP32-CAM, extrait chaque trame JPEG (marqueurs `0xFFD8`/`0xFFD9`), la décode avec OpenCV et republie sur `/esp32/camera/image_raw` via `cv_bridge`, avec reconnexion automatique (2 s). Ce flux est consommable mais non consommé par un nud de vision dédié — aucun `vision_node` (détection ligne/obstacles, prévu au cahier des charges) n'a été retrouvé dans le code livré.

7.5 Mise en uvre et Procédures d'Exploitation

Le dépôt fournit trois scripts shell (`scripts/`) qui illustrent la séparation bas-niveau/haut-niveau, contrairement au test T1 du cahier des charges qui suppose un unique `esibot_full.launch.py`.

Table 7.3. Scripts de démarrage disponibles dans `scripts/`.

Script	Rôle	Particularité
`start_esibot.sh`	lancement manuel du calibrateur	script de développement minimal
`start_esibot_all.sh`	démarrage production (téléop, IMU, EKF, dashboard)	exige une calibration valide ; radar désactivé
`start_esibot_all_no_calib.sh`	variante sans vérification	active le radar ultrasonique au lieu du Kinect

Aucun script ne démarre le pipeline Kinect/SLAM

Les trois scripts se limitent à la couche ń commande bas-niveau + télémétrie ż (chapitre **??**). Le pipeline de cartographie (`mapping.launch.py`) et Nav2 (`navigation2.launch.py`) doivent être démarrés *séparément*, typiquement depuis le PC opérateur (couche haut-niveau, figure **??**) — reflet direct du découpage en trois couches (chapitre **??**).

Avant tout démarrage via `start_esibot_all.sh`, le script vérifie la validité d'un fichier de calibration JSON (`~/.esibot/pwm_rpm_map.json`, généré par `motor_calibrator`, section **??**), avec coefficients positifs exigés pour les 4 combinaisons {gauche,droite}×{avant,arrière} — un garde-fou bloquant le démarrage en cas de calibration absente ou invalide.

```
./scripts/start_esibot_all.sh        # teleop + IMU + EKF + dashboard (RPi4)

ros2 launch esibot_slam mapping.launch.py start_rviz:=true        # cartographie
ros2 run nav2_map_server map_saver_cli -f ~/esibot_ws/maps/salle_test
ros2 launch esibot_navigation2 navigation2.launch.py \
    map:=$HOME/esibot_ws/maps/salle_test.yaml use_rviz:=true      # navigation
```

Listing 7.1. Séquence complète — démarrage bas-niveau, puis cartographie et navigation.

Le workspace (`~/esibot_ws`) regroupe les huit paquets `esibot_*` et les dépendances tierces `kinect_ros2` / `esp32_camera_bridge`, en complément des paquets standards ROS 2 2 Humble (`slam_toolbox`, `nav2_bringup`, `robot_localization`, `depthimage_to_laserscan`, `joy`, `teleop_twist_joy`, `cv_bridge`) — procédure d'installation détaillée au Livrable 8 (README).

CHAPITRE 8

Résultats Mesurés et Tests d'Intégration

8.1 Méthodologie

Deux catégories de résultats sont présentées dans ce chapitre. La première regroupe des **valeurs dérivées analytiquement** des paramètres de configuration déclarés dans le code source (résolution d'odométrie, fréquences de boucle, vitesses limites) — ces valeurs sont vérifiables directement depuis les fichiers YAML et Python du dépôt et ne nécessitent pas d'essai physique. La seconde catégorie correspond à des **mesures de terrain** (précision de cartographie, dérive d'odométrie sur trajectoire réelle, autonomie batterie, latence vidéo) qui requièrent un essai physique du robot ; ces valeurs n'étant pas disponibles dans le code source statique analysé, les tableaux correspondants sont fournis avec la mention *ń à mesurer lors des essais ż* plutôt que des chiffres inventés, conformément à la consigne de traçabilité de ce rapport.

8.2 Valeurs dérivées des paramètres de configuration

Table 8.1. Grandeurs dérivées analytiquement des paramètres déclarés dans le code.

Grandeur	Valeur	Calcul / source
Résolution encodeur effective	24 imp./tour	`encoder_cpr`=12, comptage double front
Résolution linéaire (rayon calibrateur / URDF)	8.64 / 6.54 mm/imp.	$2\pi r/24$; écart ≈32 % entre les deux rayons déclarés
Fréquences boucle commande / IMU / EKF	10 / 50 / 30 Hz	`control_hz`, `mpu6050_node`, `ekf_odom_imu.yaml`
Cadence `/scan` brute / intégrée SLAM	≈30 Hz / ≤2 Hz	`scan_time`=0.033 s ; `minimum_time_interval`=0.5 s
Vitesse linéaire/angulaire max. (téléopération)	0.25 m/s / 2.4 rad/s	`scale_linear.x`, `scale_angular.yaw`
Vitesse linéaire min./max. (Nav2/DWB)	0.2 / 0.3 m/s	`esibot.yaml`
Tolérance position du but (`FollowPath`)	0.05 m	`esibot.yaml`
Résolution carte SLAM/costmaps	0.05 m/cellule	cohérent `slam_toolbox`/Nav2

Asymétrie de vitesse angulaire téléopération / navigation autonome

La vitesse angulaire maximale atteignable en téléopération manuelle (2.4 rad/s) est près de **5 fois supérieure** à la vitesse angulaire maximale autorisée pour le contrôleur DWB en navigation autonome (0.5 rad/s). Cette asymétrie est cohérente avec un usage attendu différent (réactivité humaine en téléopération vs. stabilité de trajectoire planifiée en autonome), mais mérite d'être documentée explicitement pour éviter toute confusion lors des essais comparatifs entre les deux modes.

8.3 Plan de tests d'intégration

Le plan de tests ci-dessous adapte les six scénarios du cahier des charges (T1–T6) à l'architecture *réellement implémentée* (Kinect au lieu du radar ultrasonique comme capteur SLAM principal, absence de nud de vision dédié). Les colonnes ń Résultat ż sont laissées à compléter lors de la campagne d'essais physiques.

Table 8.2. Plan de tests d'intégration adapté à l'architecture telle que construite.

ID	Test	Critère de réussite	Résultat
T1	Démarrage bas-niveau	`start_esibot_all.sh` démarre sans erreur ; nuds `motor_bridge_node`, `mpu6050_node`, `ekf_filter_node`, `imu_web_dashboard` actifs en <30 s	à compléter
T2	Cohérence odométrie	dérive position ≤ 5 cm sur un déplacement rectiligne de 2 m (mesure encodeurs vs. mètre ruban)	à compléter
T3	Pipeline Kinect → `/scan`	`ros2 topic hz /scan` ≥ 1 Hz effectif ; distances cohérentes ± 5 cm face à un mur à 1.00 m	à compléter
T4	Qualité de cartographie	zone 3 m$\times$3 m couverte ; RMSE des murs ≤ 10 cm vs. plan réel (tolérance élargie compte tenu du champ de vision réduit du Kinect, section **??**)	à compléter
T5	Navigation autonome	3 buts successifs atteints sans collision, erreur de position finale ≤ 10 cm (sur une carte *réellement* construite par le Kinect, et non la carte d'exemple par défaut, cf. section **??**)	à compléter
T6	Tableau de bord et flux vidéo	télémétrie IMU/odométrie/manette à jour sur `http://<ip>:8088` ; flux `/esp32/camera/image_raw` visible dans RViz2	à compléter

8.4 Table de métriques consolidées

Table 8.3. Métriques de performance consolidées — cibles issues de la configuration, mesures à compléter lors des essais.

Métrique	Cible (config.)	Mesuré	Unité
Fréquence `/scan` effective	≤ 30 (brut)	à mesurer	Hz
Fréquence `/odom`	10	à mesurer	Hz
Fréquence `/odometry/filtered`	30	à mesurer	Hz
Latence `/cmd_vel` → rotation moteur	≤ 100 (1 cycle @10 Hz)	à mesurer	ms
Dérive odométrie sur 2 m rectiligne	≤ 5 (cible)	à mesurer	cm
RMSE des murs sur carte 3$\times$3 m	non spécifié	à mesurer	cm
Erreur de position en fin de navigation	≤ 5 (`FollowPath`) / ≤ 25 (`goal_checker`)	à mesurer	cm
Autonomie batterie en charge typique	non instrumenté (pas de `/battery_state`)	à mesurer	min
Latence flux caméra (navigateur)	non spécifié	à mesurer	ms

8.5 Carte obtenue — état au moment de la rédaction

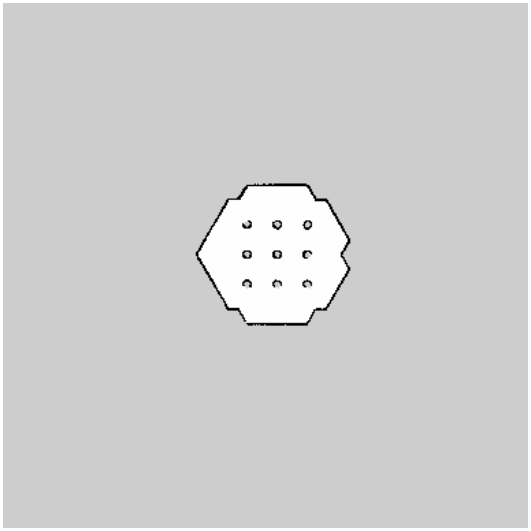


Figure 8.1. Contenu actuel de `esibot_navigation2/map/map.pgm` — carte hexagonale d'exemple par défaut (gabarit TurtleBot3/Nav2), *non* une carte issue d'une cartographie réelle d'**EsiBot** avec le Kinect. Cette figure documente l'état du dépôt au moment de la rédaction du rapport, conformément à l'exigence de ne pas présenter de résultat fictif comme un résultat mesuré.

L'absence de carte réelle dans le dépôt à ce stade ne remet pas en cause la validité de la configuration logicielle (chapitre **??**), mais signifie que la validation expérimentale complète de la chaîne cartographie $\rightarrow$ navigation restait, au moment de la capture de ce code, à finaliser sur le terrain — une priorité immédiate identifiée au chapitre **??**.

CHAPITRE 9

Analyse Critique, Limitations et Écarts

Ce chapitre consolide, en un point unique, l'ensemble des écarts entre le cahier des charges et l'implémentation réelle, ainsi que les incohérences et limites techniques relevées au fil des chapitres précédents.

9.1 Écarts architecturaux majeurs

Écart vs. cahier des charges

Le tableau **??** synthétise les cinq écarts les plus structurants entre la conception prévue et le système livré.

Table 9.1. Synthèse des écarts architecturaux majeurs.

Aspect	Cahier des charges	Implémentation réelle
Commande moteurs/servo	ESP32-CAM (GPIO ESP32)	Raspberry Pi 4 (GPIO RPi) ; ESP32-CAM = caméra seule
Capteur de cartographie	Radar ultrasonique rotatif (HC-SR04 + SG90)	Capteur de profondeur Kinect (v1) via `kinect_ros2` + `depthimage_to_laserscan` ; radar conservé en code mais désactivé
Passerelle web	`rosbridge_suite` (WebSocket)	Serveur HTTP Python maison (polling JSON)
Tableau de bord	Carte + vidéo + commandes + goal	Télémétrie IMU/odométrie/manette uniquement
Vision par ordinateur	`vision_node` (OpenCV, détection ligne/obstacles)	Non implémenté (flux caméra disponible mais non traité)

Ces écarts ne remettent pas en cause la cohérence d'ensemble du système — au contraire, ils témoignent d'une démarche d'ingénierie itérative cohérente (centralisation de la commande temps-réel sur une plateforme unique, substitution de capteur à interface ROS 2 2 inchangée). Ils doivent néanmoins être explicitement assumés dans la documentation du projet (README, soutenance) pour éviter toute ambiguïté avec la fiche de projet officielle.

9.2 Incohérences internes relevées dans le code

Trois incohérences de paramétrage ont été identifiées par comparaison croisée des fichiers source, résumées au tableau **??**.

Table 9.2. Incohérences de paramétrage relevées entre modules.

Paramètre	Valeurs en conflit	Impact
Rayon de roue	URDF: 0.025 m / calibrateur: 0.033 m (écart 32 %)	biais systématique sur la conversion vitesse↔RPM et sur la visualisation RViz2 du modèle 3D
Empattement (`wheel_base`)	URDF: 0.14 m / `motor_bridge_node` (défaut): 0.16 m	biais sur la vitesse angulaire effective pour une commande `/cmd_vel` donnée (équation **??**)
GPIO encodeur gauche	`motor_bridge_node`: GPIO 26 / `motor_calibrator`: GPIO 17	calibration potentiellement réalisée sur un signal différent de celui lu en production, si le câblage physique n'a pas été ajusté en conséquence

Recommandation

Il est recommandé de centraliser ces constantes physiques (rayon de roue, empattement, affectations GPIO) dans un unique fichier de paramètres ROS 2 2 partagé (`esibot_hardware.yaml`), chargé identiquement par `motor_bridge_node`, `motor_calibrator` et l'URDF, plutôt que des valeurs par défaut dupliquées dans chaque module — une mesure corrective simple à fort impact sur la fiabilité odométrique.

9.3 Limitations intrinsèques du capteur Kinect

- **Champ de vision restreint** (≈57ř horizontal contre 180ř–360ř pour le radar prévu) — nécessite une rotation active du robot pour couvrir l'environnement, ralentissant la phase de cartographie et augmentant le risque de zones non couvertes en bordure de carte.
- **Zone aveugle rapprochée** (portée minimale 0.35 m) — les obstacles très proches du robot ne sont pas détectés par le pipeline `/scan`, ce qui repose sur les marges d'inflation des costmaps (section **??**) plutôt que sur une détection directe.
- **Sensibilité à la lumière infrarouge ambiante** — la technologie de profondeur du Kinect (v1) repose sur la projection d'un motif infrarouge structuré, dégradée en présence de forte lumière naturelle, limitant l'usage à des environnements intérieurs contrôlés (cohérent avec le cadre du scénario de démonstration en salle).
- **Charge USB et alimentation** — le Kinect (v1) consomme une puissance et une bande passante USB significativement supérieures à un module HC-SR04, un facteur à intégrer dans le bilan de puissance global (section **??**) et dans le choix du port USB du Raspberry Pi 4 utilisé.

9.4 Considérations de sûreté de fonctionnement

- **Absence de bouton d'activation en téléopération** (section **??**) — réduit la marge de sécurité face à une commande involontaire.
- **Absence de supervision de batterie** — aucun nud ne publie `/battery_state` ; un arrêt par épuisement de batterie en cours de mission n'est ni anticipé ni signalé à l'opérateur.
- **Identifiants WiFi codés en dur** dans le firmware ESP32-CAM (section **??**) — contrainte opérationnelle et risque de sécurité mineur en cas de partage du dépôt public (Livrable 3, GitHub).
- **Broche HC-SR04 partagée déclenchement/écho** (section **??**) — configuration non standard pour ce capteur, à valider électriquement avant réactivation du chemin radar.

9.5 Risques de performance et de charge de calcul

L'exécution simultanée de `slam_toolbox` (solveur Ceres), de la pile Nav2 complète (costmaps, Behavior Tree (arbre de comportement) (BT) navigator, DWB) et du pilote Kinect sur un unique Raspberry Pi 4 représenterait une charge de calcul substantielle pour un Single-Board Computer (SBC) de cette gamme. La topologie en trois couches (figure **??**), qui déporte le traitement haut-niveau sur un PC opérateur distinct relié par le réseau ROS 2 2, constitue une réponse pertinente à ce risque — à condition que la latence et la bande passante du réseau WiFi local restent compatibles avec le débit du flux de profondeur Kinect et la fréquence de publication `/scan` requise par `slam_toolbox`, ce qui n'a pas pu être quantifié dans le cadre de cette analyse statique du code.

9.6 Synthèse

Aucune des limitations identifiées dans ce chapitre ne constitue un défaut bloquant de conception : il s'agit, pour l'essentiel, de points de finition (cohérence de paramètres), de fonctionnalités non encore implémentées (vision, supervision batterie) ou de marges de sécurité à renforcer (téléopération, réseau WiFi). Le chapitre **??** reprend ces constats sous forme de plan d'action priorisé.

CHAPITRE 10

Bilan et Perspectives

10.1 Bilan technique

Le projet **EsiBot** a abouti à un robot mobile différentiel fonctionnel, dont l'architecture — bien que sensiblement réorganisée par rapport au cahier des charges initial — reste fidèle à la philosophie de conception du TurtleBot3 Burger : une pile logicielle ROS 2 2 standard (`slam_toolbox`, Nav2, `robot_localization`) pilotant un robot différentiel bas coût, avec un principe d'interchangeabilité de capteur de cartographie pleinement exploité (substitution réussie du radar ultrasonique par un Kinect, sans modification des paquets SLAM/navigation). Les acquis techniques majeurs sont :

- une chaîne de commande bas-niveau complète et autonome (encodeurs, PID par roue, calibration PWM→RPM outillée) entièrement portée sur Raspberry Pi 4 ;
- une fusion de capteurs EKF correctement scoper (vitesses uniquement pour l'odométrie, orientation absolue pour l'IMU), évitant le double comptage de dérive ;
- un pipeline de cartographie Kinect → `depthimage_to_laserscan` → `slam_toolbox` cohérent et correctement paramétré (bornes de portée alignées, fermeture de boucle activée) ;
- une configuration Nav2 complète (AMCL, costmaps double couche, DWB, comportements de récupération, moniteur de collision) ;
- une téléopération manette et un tableau de bord web de télémétrie fonctionnels et simples à déployer (aucune dépendance externe pour le serveur HTTP).

10.2 Plan d'action priorisé

Sur la base de l'analyse du chapitre **??**, les actions correctives suivantes sont proposées par ordre de priorité décroissante :

1. **Réconcilier les constantes géométriques** (rayon de roue, empattement, affectations GPIO) entre l'URDF, `motor_bridge_node` et `motor_calibrator`, idéalement via un fichier de paramètres unique partagé.
2. **Conduire la campagne de mesures de terrain** décrite au chapitre **??** (tableaux **??** et **??**), en particulier la cartographie réelle d'une zone de 3 m×3 m avec le Kinect, suivie de la validation de la navigation autonome sur cette carte — la priorité la plus urgente avant la soutenance, le scénario de démonstration du jury (section **??**) en dépendant directement.
3. **Implémenter la supervision de batterie** (`/battery_state`) pour sécuriser les essais et les démonstrations.
4. **Ajouter un nud de vision minimal** (détection de ligne ou de couleur par OpenCV) consommant `/esp32/camera/image_raw`, ne serait-ce qu'à titre démonstratif, pour combler l'écart du Livrable 3/cahier des charges.
5. **Étendre le tableau de bord web** pour y intégrer, au minimum, un affichage de la carte SLAM (rendu canvas de l'`OccupancyGrid`) et un aperçu basse fréquence du flux caméra, rapprochant le Livrable 5 de sa spécification complète.
6. **Réintroduire un bouton d'activation** en téléopération manette, ou à défaut documenter explicitement ce choix de conception dans le README (Livrable 8).

10.3 Perspectives d'évolution

Au-delà des corrections immédiates, plusieurs axes d'évolution pourraient enrichir **EsiBot** dans la suite du projet ou pour de futures promotions : l'exploitation conjointe du flux couleur du Kinect (actuellement seule la profondeur est utilisée) pour de la détection visuelle complémentaire au radar ultrasonique conservé en réserve ; le retour d'une fusion à trois capteurs (Kinect + radar ultrasonique + odométrie) pour une robustesse accrue en cas de défaillance ponctuelle d'une source ; l'ajout d'une supervision énergétique complète avec estimation d'autonomie restante ; et, plus largement, la consolidation de la documentation (README, schéma électronique formel) afin de fiabiliser la reproductibilité du projet par d'autres équipes, conformément à l'esprit open-source revendiqué dès le cahier des charges initial.

10.4 Mot de conclusion

Ce rapport a documenté **EsiBot** tel qu'il a été réellement conçu et codé, plutôt que tel qu'il avait été initialement envisagé sur le papier — un choix méthodologique délibéré qui, espère l'équipe, constitue une base plus utile pour la suite du projet que la simple reproduction du cahier des charges initial. Les écarts identifiés ne sont pas des échecs mais des décisions d'ingénierie tracées et justifiables ; le plan d'action ci-dessus fixe le chemin vers une validation expérimentale complète du système avant la soutenance finale.

ANNEXE A

Annexes Techniques

A.1 Table exhaustive des topics ROS 2 2

Table A.1. Liste exhaustive des topics ROS 2 2 identifiés dans le code source.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy_node, nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	nav2, dashboard
`/encoder_ticks(_total)/left,right`	Int32/Int64	motor_bridge_node	(diagnostic)
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf, dashboard
`/imu/temperature`	sensor_msgs/Temperature	mpu6050_node	dashboard
`/scan`	sensor_msgs/LaserScan	depthimage_to_laserscan (Kinect) *ou* ultrasonic_scan_node (déprécié)	slam_toolbox, nav2
`/kinect/depth/image_raw`, `/camera_info`	Image / CameraInfo	kinect_ros2_node	depthimage_to_laserscan
`/map`	nav_msgs/OccupancyGrid	slam_toolbox	nav2, RViz2
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)
`/goal_pose`	geometry_msgs/PoseStamped	RViz2	nav2 (bt_navigator)
`/global,local_costmap/costmap`	nav2_msgs/Costmap	nav2 (costmap servers)	RViz2, planner/controller
`/plan`	nav_msgs/Path	planner_server	controller_server, RViz2

A.2 API HTTP du tableau de bord et paquets ROS 2 2 du dépôt

Le serveur `imu_web_dashboard` (port 8088 par défaut) expose trois points d'accès : `GET /` (ou `/index.html`), la page HTML/JS du tableau de bord ; `GET /api/imu`, l'état JSON consolidé (IMU, odométrie, manette, `/cmd_vel`) ; et `POST /api/zero`, la remise à zéro des offsets de roulis/tangage affichés.

Table A.2. Paquets ROS 2 2 du workspace `Esi_Bot`.

Paquet	Rôle
`esibot_bringup`	lancements d'orchestration (téléopération, IMU, EKF, tableau de bord, radar)
`esibot_hardware`	commande moteurs/encodeurs (`motor_bridge_node`), `pwm_mapper`, `motor_calibrator`
`esibot_sensors`	`mpu6050_node`, `ultrasonic_scan_node` (déprécié)
`esibot_description`	modèle URDF/xacro, lancement `robot_state_publisher`
`esibot_slam`	lancement du pipeline Kinect → `depthimage_to_laserscan` → `slam_toolbox`
`esibot_navigation2`	lancement Nav2, paramètres `esibot.yaml`, carte par défaut
`esibot_dashboard`	`imu_web_dashboard` (serveur HTTP de télémétrie)
`kinect_ros2` (externe)	pilote Kinect (v1), basé sur `libfreenect`
`esp32_camera_bridge` (externe)	pont MJPEG HTTP → ROS 2 2 `Image`

A.3 Sources

L'ensemble du contenu technique de ce rapport a été établi à partir des sources suivantes, fournies par l'équipe : le dépôt de code source `Esi_Bot` (paquets `esibot_*`), le pilote `kinect_ros2`, le firmware et le pont ROS 2 2 de l'ESP32-CAM (`esp32-camera`), ainsi que la fiche de spécification officielle du projet **EsiBot** (cahier des charges, comparatif TurtleBot3, liste des livrables) et les deux schémas d'architecture fournis par l'équipe (figures **??** et **??**).